Доклад: «Тестирование крупномасштабных финансовых систем: опыт платформы цифрового рубля»

Уважаемые коллеги!

Сегодня мы поговорим о том, как организовать и провести два критически важных вида тестирования в крупном, распределённом проекте — нагрузочное тестирование и тестирование устойчивости к отказам. В качестве примера мы расскажем о нашем опыте работы с платформой цифрового рубля (ПлЦР) — сложной, многоуровневой системой, интегрированной с внешними сервисами, включая Национальную систему платёжных карт (НСПК).

Таблица 1

login	user1
password	password1

Наш доклад — это взгляд изнутри: от команды разработки и тестирования, которая участвовала в создании инструментов и проведении испытаний. Мы поделимся тем, какие вызовы мы встретили, какие решения приняли, и какие выводы сделали.

1. Почему обычное тестирование недостаточно?

Платформа цифрового рубля — это не просто приложение. Это экосистема, включающая:

внутренние компоненты Банка России,
интеграцию с НСПК,
взаимодействие с банками и пользователями,
строгие требования к безопасности, надёжности и производительности.

В таких условиях недостаточно просто проверить, «работает или нет». Нужно убедиться, что система:

выдерживает пиковую нагрузку (миллионы транзакций в час),
корректно обрабатывает ошибки и сбои,
сохраняет целостность данных даже при отказе отдельных компонентов.

Именно поэтому мы сосредоточились на двух направлениях:

Нагрузочное тестирование — проверка производительности под реалистичной нагрузкой.

Тестирование устойчивости к отказам — моделирование сбоев и проверка реакции системы.

2. Проблема: нет реальной среды для тестирования

Одна из главных сложностей — отсутствие доступа к реальным внешним системам на этапе тестирования. Например, для проверки взаимодействия ПлЦР с НСПК по схеме C2B (клиент-бизнес) нужен Сервис управления ссылками (СУС). Но СУС — это часть НСПК, и подключать его напрямую к тестовому стенду невозможно.

Решение: мы разработали специализированный эмулятор СУС.

Это не просто «заглушка», а полноценный сервис, который:

принимает JMS-сообщения,
расшифровывает XML-запросы,
определяет тип платёжной ссылки (статическая, динамическая, кассовая),
генерирует корректные ответы — включая ошибки и негативные сценарии.

Такой эмулятор позволяет тестировать ПлЦР в полной изоляции, но с сохранением всех бизнес-правил и протокольных требований.

3. Как мы делали эмулятор?

Мы пошли на компромисс между гибкостью и производительностью.

Требования:

Должен работать и с GUI (для тестировщиков),
И в консольном режиме (для автоматизации и серверов),
Поддерживать Windows и Linux.
Технологии:

Java + Spring Boot — для серверной логики, JMS, базы данных.
JavaFX — для графического интерфейса.
Модульная система Java — чтобы чётко разделить фронтенд и бэкенд.

Это дало нам гибкость: тестировщики запускают эмулятор через интерфейс, а CI/CD-процессы — через консоль. Но потребовало аккуратной настройки: два файла конфигурации (pom.xml и module-info.java), отладка рефлексии и т.д.

4. От отладки — к нагрузке

Чтобы протестировать сам эмулятор, нам понадобился генератор трафика. Мы построили его поэтапно:

Файловый провайдер — читал готовые XML-сообщения из файлов. Позволил отладить логику обработки.

Генератор одиночных сообщений — отправлял запросы через JMS-брокер. Проверили интеграцию с очередями.

Нагрузочный сервис-имитатор — полноценный Spring Boot-сервис, который:

генерирует валидные и невалидные запросы,

моделирует разные типы нагрузки (равномерную, пиковую),

поддерживает полный цикл транзакций.

Этот инструмент стал ключевым компонентом для последующего нагрузочного тестирования ПлЦР.

5. Нагрузочное тестирование: как мы это делаем

Для ПлЦР мы разработали специальный податчик нагрузки — ПН.

Что он делает:

Формирует электронные сообщения (ЭС) на основе шаблонов,
Подписывает их через КС «Янтарь»,
Отправляет в тестовый шлюз КБР,
Собирает ответы и считает метрики.
Важно: вся информация — тестовая, никаких реальных данных.

Архитектура ПН:

Контроллер — управляет процессом,
Генератор — создаёт сообщения,
Податчик — отправляет их с заданной интенсивностью,
Всё связано через брокер сообщений (Factor-ESB).
Методика:

Нагрузка подаётся поэтапно: сначала низкая, потом пиковая.
После каждого этапа — пауза на стабилизацию.
Измеряются:
TPS (транзакций в секунду),
Время обработки каждого сообщения,
Доля ошибок и потерянных сообщений.
Критерий успеха:

99,9% сообщений должны быть обработаны в рамках нормативного времени.

6. Устойчивость к отказам: как проверяем?

Здесь эмулятор СУС играет ключевую роль. Мы можем:

Отключить эмулятор в середине теста — проверить, как ПлЦР реагирует на недоступность внешнего сервиса.

Вернуть ошибку вместо ответа — проверить обработку исключений.

Замедлить ответ — смоделировать сетевые задержки.

Такие сценарии помогают убедиться, что система не «падает», а корректно переходит в режим деградации, сохраняя данные и логируя ошибки.

7. Результаты и выводы

Эмулятор СУС стал универсальным инструментом:

Используется и в функциональном тестировании,

И в нагрузочных испытаниях,

И на разных стендах — от разработки до приёмки.

Нагрузочное тестирование показало:

Где «узкие места» (БД, JMS, однопоточность),

Как их устранить (пулы соединений, кэширование, многопоточность).

Производительность выросла:

С 4 до 60 обработанных сообщений в секунду — только за счёт оптимизации архитектуры.

Организационный эффект:

Улучшено взаимодействие между разработкой, тестированием и эксплуатацией,

Команда РЦТиКК получила глубокую экспертизу в Spring, JMS, нагрузочном тестировании.

Заключение

Тестирование крупных финансовых систем — это не просто запуск автотестов. Это инженерная дисциплина, требующая:

Глубокого понимания бизнес-логики,

Создания специализированных инструментов,

Тесного взаимодействия между командами.

Наш опыт показывает: даже в условиях ограниченного доступа к внешним системам можно построить реалистичную, масштабируемую и контролируемую тестовую среду. А это — залог стабильности и надёжности в промышленной эксплуатации.

Спасибо за внимание! Готовы ответить на ваши вопросы.